IS 1210
Database Systems

Lecture 4 – Structured Query Language (SQL) –Part 2

MVPT Lakshika,

Lecturer (Probationary),

Department of Information Systems Engineering, UCSC.

Contact: tlv@ucsc.cmb.ac.lk

Outline of this session…

1. ISO SQL Data Types (Cont…)
2. ISO SQL Scalar Operators

ISO SQL Data types

- The basic **data types** available for attributes include:
  - Numeric
  - String (Character string & Bit string)

  Discussed in Lecture 4 – Part 1

  - **Boolean**
  - **Temporal**
- Within each of these general types may be several subtypes.

  e.g., numeric data types: Exact Numeric and Approximate Numeric

ISO SQL Data types - Boolean data

- **Boolean** data consists of the distinct truth values **TRUE** and **FALSE**.
- In SQL, TRUE is considered **greater than** FALSE (TRUE > FALSE).
- Unless prohibited by a NOT NULL constraint, Boolean data also supports the **UNKNOWN** truth value as the NULL value.

e.g., Analogy**:** "Is it raining outside?“ **True:** You see rain, **False:** You see the sun, **Unknown:** You are in a room with no windows.

e.g., CREATE TABLE user_accounts (user_id SERIAL PRIMARY KEY, ….

is_active BOOLEAN,

is_verified BOOLEAN NOT NULL DEFAULT FALSE,

marketing_opt_in BOOLEAN DEFAULT NULL);

- Any comparison involving the NULL value or an UNKNOWN truth value returns an UNKNOWN result.

ISO SQL Data types – Temporal Data

- The **datetime data** type is used to define points in time to a certain degree of accuracy.
- The ISO standard subdivides the datetime data type into YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, TIMEZONE_HOUR, and TIMEZONE_MINUTE.
  - The latter two fields specify the hour and minute part of the **time zone** offset from Universal Coordinated Time (which used to be called Greenwich Mean Time).
- Three main types:

```
DATE
TIME [timePrecision] [WITH TIME ZONE]
TIMESTAMP [timePrecision] [WITH TIME ZONE]
```

- **[timePrecision]** is the number of decimal places of accuracy to which the SECOND field is kept.
- If not specified,
  - **TIME** defaults to a precision of **0** (that is, whole seconds)
  - **TIMESTAMP** defaults to **6** (that is, microseconds)

ISO SQL Data types – Temporal Data

- **DATE** data type is used to store calendar dates.

  e.g., viewDate **DATE**

  - Components are YEAR, MONTH, and DAY in the form ***YYYY-MM-DD*** (ten positions) (e.g., '2014-09-27')

- **TIME** data type is used to store time.

  e.g., loginTime **TIME**

  - Components are HOUR, MINUTE, and SECOND in the form ***HH:MM:SS*** (eight positions) (e.g., '09:12:47')

- **TIMESTAMP** is used to store both dates and times.

  e.g., logTime **TIMESTAMP**

  - Components are **DATE** and **TIME** fields, plus a minimum of six positions for decimal fractions of seconds (e.g., '2014-09-27 09:12:47.648302')

ISO SQL Data types - Temporal Data

- Only valid dates and times should be allowed by the SQL implementation.

  e.g., months should be between 1 and 12, days must be between 1 and 31
- The < (less than) comparison can be used with dates or times.
  - An earlier date is considered to be smaller than a later date, and similarly with time.
- Literal values are represented by single-quoted strings preceded by the keyword.

  e.g., CREATE TABLE *Employee* (viewDate **DATE**);

  INSERT INTO *Employee* (viewDate) VALUES (‘2014-09-27‘);

  CREATE TABLE *Employee* (loginTime **TIME**);

  INSERT INTO *Employee* (loginTime) VALUES (‘09:12:47’);

ISO SQL Data types - Temporal Data

- **INTERVAL** data type is used to represent periods of time.
  - MySQL does not have a built-in INTERVAL data type.
  - But supports INTERVAL in expressions to manipulate dates and times.
- Two classes of interval data type:
  - The **year–month** class may contain only the YEAR and/ or the MONTH fields (YYYY-MM)
  - The **day–time** class may contain only a contiguous selection from DAY, HOUR, MINUTE, SECOND (DD HH:MM:SS).

```
INTERVAL {{startField TO endField} singleDatetimeField}
startField = YEAR | MONTH | DAY | HOUR | MINUTE
             [(intervalLeadingFieldPrecision)]
endField   = YEAR | MONTH | DAY | HOUR | MINUTE | SECOND
             [(fractionalSecondsPrecision)]
singleDatetimeField = startField | SECOND
             [(intervalLeadingFieldPrecision [, fractionalSecondsPrecision])]
```

ISO SQL Data types - Temporal Data

- In all cases, *startField* has a leading field precision that defaults to 2.

  e.g., **INTERVAL YEAR**(2) **TO MONTH**

  - represents an interval of time with a value between 0 years 0 months, and 99 years 11 months (e.g., "2 years and 6 months“)

  e.g., **INTERVAL HOUR TO SECOND**(4)

  - represents an interval of time with a value between 99 hours 59 minutes 59.9999 seconds (the fractional precision of second is 4).

- If the precision is **not explicitly specified**, the default behavior is determined by the database implementation, which is commonly **2 digits**.
- The most powerful feature of Intervals is that you can add or subtract them directly from dates.

ISO SQL Data types - Summary

- The ISO standard provides eight base data types.

Data type	Declarations				
boolean	BOOLEAN				
character	CHAR	VARCHAR			
bit†	BIT	BIT VARYING			
exact numeric	NUMERIC	DECIMAL	INTEGER	SMALLINT	BIGINT
approximate numeric	FLOAT	REAL	DOUBLE PRECISION		
datetime	DATE	TIME	TIMESTAMP		
interval	INTERVAL				
large objects	CHARACTER LARGE OBJECT		BINARY LARGE OBJECT		

† BIT and BIT VARYING have been removed from the SQL:2003 standard.

ISO SQL SCALAR Operators

- SQL provides several built-in scalar operators and functions that can be used to construct a scalar expression (an expression that evaluates to a **scalar or a single value**)
- Scalar operators:

  Arithmetic operators: +, −, *, /

  Comparison: =, >, <, !=

  Concatenation: ||

  Logical: AND, OR, NOT

  e.g.,
- Some other operators:

  CHAR_LENGTH: Returns the length of a string in characters. e.g., **CHAR_LENGTH**('Beech') = 5

  LOWER, UPPER, CURRENT_DATE, CURRENT_TIME, TRIM, POSITION, SUBSTRING, CURRENT_DATE, CURRENT_TIME, POWER, SQRT, FLOOR, CEIL …

Commenting

- Comments in SQL are similar to comments in other programming languages, such as Java, C++, Python, etc.
- They are primarily used to define a code section for easier understanding.
- Comments can be,

  - Single line

```
--Select all:
SELECT * FROM Customers;
```

  - Inline

```
SELECT * FROM Customers -- WHERE City='Berlin';
```

  - Multi-line

```
/*Select all the columns
of all the records
in the Customers table:*/
SELECT * FROM Customers;
```

Any questions?